

SMARTVISION

AI-Based Object Detection, Tracking & Counting System

Computer Vision — Evaluated Project

Student: _____

Registration No.: _____

Course: Computer Vision

Academic Year: 2026

1. Introduction

SmartVision is a modular command-line Computer Vision application for processing images and videos. It combines object detection, video tracking, counting and result analytics in one workflow. The project is designed to demonstrate practical application of Computer Vision concepts while remaining executable from a terminal.

2. Problem Statement

Manual inspection of visual data can require significant time when many objects or video frames must be reviewed. SmartVision addresses this by automating object detection and providing annotated media and structured count summaries.

3. Functional Requirements

FR1 — Image detection: accept an image, detect objects and save an annotated image.

FR2 — Video tracking: process video frames and use persistent tracking IDs where supported by the model pipeline.

FR3 — Object counting: count detected classes across processed frames and export a CSV summary.

FR4 — Validation: reject missing or invalid input paths and invalid confidence values.

FR5 — Analytics export: save class/count pairs in CSV format.

4. Non-Functional Requirements

Performance: avoid unnecessary GUI operations and process media directly from files.

Usability: expose a small command-line interface with explicit modes.

Reliability: validate input paths and report processing errors.

Maintainability: separate detection, tracking, counting, analytics and utilities into modules.

Resource efficiency: write processed frames incrementally instead of storing an entire video in memory.

Testability: provide automated tests for utilities and result serialization.

5. System Architecture

The architecture follows a modular pipeline:

Command Line Interface → Input Validation → Detection Engine → Tracking/Counting Module → Output Writer → Analytics CSV.

The YOLO model is encapsulated by ObjectDetector, while video-specific operations are separated into VideoTracker and ObjectCounter.

6. Design Diagrams

Use Case: A user selects an operation, supplies an input file and receives annotated media and/or analytics.

Workflow: Start → parse arguments → validate input → load model → process image/video → write output → save CSV → finish.

Sequence: User → main.py → ObjectDetector → YOLO/OpenCV → output writer → analytics.

Component view: CLI, Detector, Preprocessing, Tracker, Counter, Analytics, File Utilities.

7. Design Decisions & Rationale

YOLO was selected because it provides a practical pretrained object-detection pipeline and supports video tracking workflows. OpenCV was selected for media input/output and frame handling. A command-line interface was chosen to satisfy the requirement that the project be executable from a terminal without GUI-dependent setup. CSV was selected for simple, portable summaries.

8. Implementation Details

The implementation uses Python classes and modules. ObjectDetector handles model loading, prediction and image annotation. VideoTracker invokes persistent model tracking for videos. ObjectCounter performs frame-level detection and aggregates class counts. Analytics writes CSV summaries. File utilities centralize path validation and directory creation.

9. Dataset Description

The application accepts local image/video files rather than requiring a single fixed dataset. For evaluation, the student should provide a small representative sample and record its source, file type, resolution, number of images/videos, and any licensing or attribution information. The repository currently contains an instruction placeholder rather than a copyrighted dataset.

10. Model Selection Rationale

A small YOLO model is used as the default to balance detection capability and local execution requirements. The model is configurable through the `--model` argument. The confidence threshold is also configurable to allow controlled experiments.

11. Evaluation Methodology

For a reproducible evaluation, run the same sample inputs with a fixed model and confidence threshold. Record whether expected object classes are detected, whether output media is created successfully, and whether CSV summaries are produced. If a labeled dataset is added, standard detection metrics such as precision, recall and mAP can be calculated from the labels.

12. Screenshots / Results

The repository includes an illustrative placeholder image to show where a final screenshot can be placed. Before submission, replace it with a real screenshot produced by the student's own execution and label the input, model and configuration used.

13. Testing Approach

Pytest tests cover input validation, directory creation and CSV serialization. Additional manual validation should confirm image processing, video processing, output file creation, corrupted-input handling and command-line argument validation.

14. Challenges Faced

Typical implementation challenges include handling different media formats, model-weight availability, varying video codecs, computational cost and maintaining a clean separation between detection and video-specific processing.

15. Learnings & Key Takeaways

The project demonstrates how pretrained computer vision models can be integrated into an application, how frame-based video processing works, how modular software design improves maintainability, and how

automated tests and command-line interfaces improve reproducibility.

16. Future Enhancements

Potential extensions include a web dashboard, configurable region-of-interest counting, line-crossing counts, per-object trajectories, database storage, benchmark reports, custom model training and a richer evaluation suite.

17. References

Ultralytics YOLO documentation and package resources.

OpenCV documentation.

Python documentation.

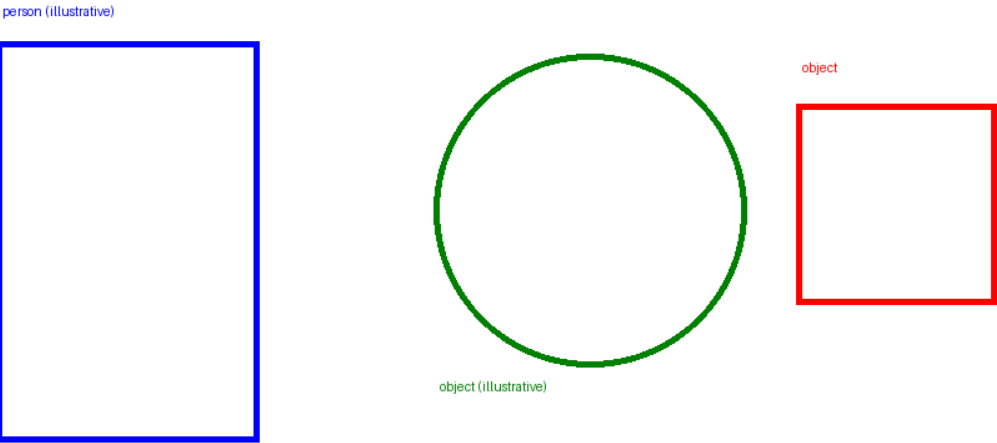
Pytest documentation.

Any dataset or sample media used in the final evaluation should be cited here with its original source and license.

Architecture Summary

Component	Responsibility
CLI / main.py	Arguments, mode selection, orchestration
ObjectDetector	YOLO loading, inference, image annotation
VideoTracker	Video tracking pipeline
ObjectCounter	Frame-level detection and aggregation
Analytics	CSV result generation
Utilities	Validation, directories and logging

Illustrative Output



Replace with an actual model-generated result before submission.